Analysis Report

- CIFAR-10 Image Classification using Deep Learning — Master's Research Project
- Contents
- 1. Research question and scope
- 2. Methodology
- 3. Results
- 4. Feature analysis
- 5. Parameter-modification study
- 6. Challenge experiments
- 7–9. Written analysis
  - 9.1 Project Execution Questions

CIFAR-10 Image Classification using Deep Learning — Master's Research Project

Author: Jyothi Kodenkiri

Programme: Master of Science — Artificial Intelligence & Machine Learning (research project)

Generated: 2026-09-08 13:30 UTC

Contents

1. Research question and scope
2. Methodology
3. Results
4. Feature analysis
5. Parameter-modification study
6. Challenge experiments
7. **Answers — 10 Project Execution Questions**
8. **Answers — 5 Parameter Modification Questions**
9. Trade-offs, reproducibility, limitations, ethics and future work
10. Appendices

---

1. Research question and scope

> Given a fixed 60,000-image, 10-class benchmark and a constrained compute budget, which convolutional architecture — a small network trained from scratch, or a large ImageNet backbone used as a frozen feature extractor — delivers the best accuracy-per-unit-cost, and what do the learned representations tell us about **why**?

Four architectures are implemented, trained on the full 40,000/10,000 split and evaluated once on the 10,000-image held-out test set. A controlled five-factor ablation study, a feature analysis and five advanced ("challenge") experiments complete the picture.

2. Methodology

2.1 Environment

Item	Value
python_version	3.9.6
platform	Darwin 25.5.0 (arm64)
processor	arm
tensorflow_version	2.16.2
keras_version	3.10.0
numpy_version	1.26.4
pandas_version	2.3.3
gpu_devices	['/physical_device:GPU:0']
gpu_available	True
num_cpus	10
seed	42
mixed_precision_policy	float32

2.2 Data

CIFAR-10 was obtained from `https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz`, extracted, and converted to the required flat CSV layout — `train.csv` with 50,000 rows × 3,073 columns and `test.csv` with 10,000 rows × 3,073 columns (1 label + 3,072 pixels). Both were verified against the study protocol programmatically.

The classes are exactly balanced (5,000 train / 1,000 test images each), which is why plain accuracy is a defensible headline metric; weighted and macro F1 are reported alongside.

2.3 Preprocessing pipeline

1. Scale to `[0,1]` (÷255).
2. Fixed-seed shuffle **before** splitting — `ImageDataGenerator(validation_split=0.2)` cuts by array index, so pre-shuffling turns that cut into a reproducible, class-balanced hold-out.
3. Resize 32×32 → 224×224 bilinear, implemented as the **first layer of every model** so the upsample runs on the GPU per batch. Materialising the resized dataset in host memory would require ≈ 33.6 GB; keeping it at 32×32 costs 0.69 GB.
4. Model-specific standardisation: per-channel mean/variance from the **training subset only** for the Simple CNN; each backbone's own `preprocess_input` for the transfer models.

Measured training-subset statistics — mean `[0.49138, 0.48219, 0.44649]`, std `[0.2473, 0.2437, 0.2616]` — match the canonical CIFAR-10 values, confirming the CSV round-trip preserved the data exactly.

2.4 Data-leakage control

#	Potential leak	Control	Verified
1	Test images seen during training	separate file; touched only by the single final evaluation	0 duplicate images between pools
2	Train/validation overlap	`validation_split` partitions by index — disjoint by construction	index intersection = 0
3	Normalisation statistics from all data	statistics from the training subset only, frozen into the model	frozen stats differ from pooled stats
4	Augmented validation images	augmentation applied after the split, to the training iterator only	two generator objects, one split
5	Test set used for model selection	callbacks monitor validation metrics exclusively	callback configuration
6	Repeated test evaluation	each model evaluated on the test set exactly once	one `evaluate_model` call per model

2.5 Architectures

- **Model 1 — Simple CNN (from scratch):** `Conv2D(32)→Pool → Conv2D(64)→Pool → Conv2D(128)→Pool → Flatten → Dense(128,ReLU) → Dropout(0.5) → Dense(10,softmax)`.
- **Models 2–4 — VGG16 / ResNet50 / EfficientNetB0:** ImageNet weights, `include_top=False`, **all backbone layers frozen**, identical custom head `GlobalAveragePooling2D → Dense(128,ReLU) → Dropout(0.5) → Dense(10,softmax)`. The base is called with `training=False` so BatchNormalization uses ImageNet running statistics rather than recomputing them from 32-image CIFAR batches.

A design conflict, handled explicitly. The study design requires both a 224×224 input and a ~500K–1M-parameter Simple CNN. These are arithmetically incompatible for the prescribed `Flatten → Dense(128)` head: three 2×2 pooling stages reduce 224 to 28, so `Flatten`

emits 28×28×128 = 100,352 units and that one Dense layer costs 12.85M parameters — 99.3 % of the network. The ~500K–1M envelope corresponds to the same architecture fed at CIFAR's native 32×32 (356K parameters). Rather than silently violate either constraint, **both variants were trained and are reported**: `Simple_CNN` at 224×224 (used in the headline four-model comparison, honouring the uniform-pipeline requirement) and `Simple_CNN_Native32` at 32×32 (honouring the parameter budget).

2.6 Training protocol

Adam (lr 0.001), categorical cross-entropy, batch 32, up to 15 epochs, with `EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True)` and `ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6)`, plus `ModelCheckpoint` and `CSVLogger` for auditability. The two callbacks deliberately monitor different quantities: validation loss degrades earlier than validation accuracy, so the scheduler always gets a chance to rescue a plateau before early stopping fires.

3. Results

3.1 Headline comparison (10,000 unseen test images)

Model	Test Acc	F1 (w)	F1 (macro)	Top-5	Params	Trainable	Epochs	Train (min)	ms/img	Size (MB)
ResNet50_Transfer	0.9056	0.9051	0.9051	0.9976	23,851,274	263,562	15	94.5	7.409	91.0
EfficientNetB0_Transfer	0.8974	0.8971	0.8971	0.9970	4,214,829	165,258	15	58.8	4.444	16.1
VGG16_Transfer	0.8441	0.8429	0.8429	0.9942	14,781,642	66,954	15	143.8	9.813	56.4
Simple_CNN	0.5533	0.5476	0.5476	0.9472	12,939,722	12,939,722	15	44.0	0.922	49.4
Simple_CNN_Native32	0.7315	0.7229	0.7229	0.9798	356,810	356,810	15	5.6	0.115	1.4

Best-performing model: ResNet50_Transfer (0.9056 accuracy, 0.9051 weighted F1).

3.2 Training behaviour and stopping conditions

Model	Epochs run	Early stopped	Best epoch	Best val acc	LR start	LR end	LR reductions	Train-val gap
ResNet50_Transfer	15	no	15	0.9136	1.0e-03	2.0e-04	1	-0.0588
EfficientNetB0_Transfer	15	no	15	0.9024	1.0e-03	1.0e-03	0	-0.0812
VGG16_Transfer	15	no	14	0.8467	1.0e-03	2.0e-04	1	-0.0637
Simple_CNN	15	no	13	0.5557	1.0e-03	4.0e-05	2	-0.1086
Simple_CNN_Native32	15	no	13	0.7363	1.0e-03	2.0e-04	1	-0.0550

3.3 Per-class F1

Class	ResNet50_Transfer	EfficientNetB0_Transfer	VGG16_Transfer	Simple_CNN
airplane	0.9131	0.9100	0.8670	0.5884
automobile	0.9479	0.9413	0.9125	0.6628
bird	0.8847	0.8715	0.7873	0.4158
cat	0.8173	0.8062	0.7326	0.3136
deer	0.8881	0.8788	0.7996	0.3883
dog	0.8707	0.8627	0.7994	0.5076
frog	0.9299	0.8977	0.8551	0.6439
horse	0.9306	0.9221	0.8603	0.6413
ship	0.9354	0.9419	0.9125	0.6867
truck	0.9337	0.9391	0.9032	0.6276

4. Feature analysis

Features are tapped from the last convolutional layer of each network — the last tensor that is still spatial and still convolutional, immediately before global pooling and classification. The layer *names* differ per architecture; the selection *principle* is identical, which is what makes the comparison fair. Analysis uses a class-balanced 2,000-image slice of the test set.

Model	Layer	Map	Raw dims	Pooled	Sparsity %	Act mean	Act std	Silhouette	2-D 10-NN	Linear probe
ResNet50_Transfer	conv5_block3_out	7×7×2048	100,352	2,048	83.33	0.472	1.765	0.0953	0.7610	0.8475
EfficientNetB0_Transfer	top_activation	7×7×1280	62,720	1,280	0.00	0.090	0.823	0.1361	0.7720	0.8525
VGG16_Transfer	block5_pool	7×7×512	25,088	512	87.61	1.221	4.866	-0.0235	0.6095	0.7515
Simple_CNN	conv3_128	56×56×128	401,408	128	88.42	0.232	1.082	-0.1339	0.2515	0.4485

5. Parameter-modification study

Every variant is trained on an identical fixed-seed 8,000-image training slice (2,000 validation) for a fixed 6 epochs and evaluated on the full 10,000-image test set. Data, seed, split and epoch budget are held constant and exactly one factor changes per comparison, so **relative conclusions are controlled and valid while absolute accuracies are not comparable to §3**.

Run	Test Acc	F1 (w)	Params	s/epoch	Overfit gap	Train MB
q3_dropout_0.5	0.8760	0.8757	4,214,829	45.1	-0.0890	3,158
q3_dropout_0.3	0.8728	0.8726	4,214,829	44.7	-0.0762	3,158
q3_dropout_0.8	0.8668	0.8663	4,214,829	45.1	-0.1614	3,158
q4_vgg_imgsize_128	0.8014	0.7989	14,781,642	39.5	-0.1211	683
q4_vgg_imgsize_224	0.7924	0.7917	14,781,642	112.7	-0.1099	1,972
q4_imgsize_128	0.4946	0.4752	4,288,970	14.8	-0.1019	222
q2_lr_0.0001	0.4903	0.4683	12,939,722	35.8	-0.1170	676
q1_batch_64	0.4524	0.4418	12,939,722	32.7	-0.1104	1,154
q1_batch_32	0.4514	0.4233	12,939,722	35.2	-0.0834	676
q2_lr_0.001	0.4514	0.4233	12,939,722	35.3	-0.0834	676
q4_imgsize_224	0.4514	0.4233	12,939,722	35.5	-0.0834	676
q5_depth_base	0.4514	0.4233	12,939,722	35.7	-0.0834	676
q5_depth_plus4	0.4422	0.4154	6,812,362	38.5	-0.1004	607
q4_imgsize_384	0.4322	0.4048	37,843,402	101.5	-0.0970	1,982
q1_batch_16	0.4135	0.3994	12,939,722	50.2	-0.0859	437
q2_lr_0.01	0.1000	0.0182	12,939,722	34.9	-0.0027	676

6. Challenge experiments

6.1 Fine-tuning

Unfreezing ResNet50_Transfer's `conv5_` stage (22 layers, BatchNormalization kept frozen) and retraining for 5 epochs at lr 1e-05 moved trainable parameters from 263,562 to 15,217,034 and test accuracy from 0.9056 to 0.9334 (**+0.0278**).

6.2 Cross-validation

3-fold stratified CV of the Simple CNN over 12,000 images: validation accuracy **0.4228 ± 0.0084** (95 % CI [0.4134, 0.4323]); test accuracy 0.4270 ± 0.0104. This sets the scale below which model-to-model differences should not be treated as meaningful.

6.3 Ensembles

Best strategy **`top2_soft_voting`** at 0.9219 (+0.0163 versus the best single model, ResNet50_Transfer at 0.9056). Gains are bounded by error correlation between members — the notebook quantifies this with a Jaccard error-overlap matrix and an ensemble ceiling.

6.4 Compression

TFLite float32 / float16 / dynamic-range int8 conversion and post-training magnitude pruning of the dense head were measured for size, accuracy and single-image CPU latency (notebook §8.4).

6.5 Real-time inference

Model	batch1_p50_ms	batch1_p95_ms	batch1_fps	batch32_throughput_ips	model_size_mb	test_accuracy
Simple_CNN	3.46543799999921	5.938602700002349	268.198871442481	1455.8919995429092	49.361122131347656	0.5533
VGG16_Transfer	9.993083000004788	14.440950000005515	93.79698131045966	132.076914417918	56.387489318847656	0.8441
ResNet50_Transfer	15.962500000000546	20.154900300001085	60.4384308452884	145.01795522251822	90.98538970947266	0.9056
EfficientNetB0_Transfer	16.024479000002145	18.427475000002143	62.042390758806555	180.6815408082235	16.078296661376953	0.8974

Lowest batch-1 latency: **Simple_CNN** at 3.47 ms (p50).

7–9. Written analysis

9.1 Project Execution Questions

Q1 — Environment and Setup

> *What GPU memory requirements would you estimate for training all four models simultaneously? How would you modify the batch size if you encounter out-of-memory errors?*

Estimating the requirement. Training memory decomposes into four terms, all measured in §3.2 by `estimate_activation_memory`:

1. **Activations** — every layer output must be retained for the backward pass. This term scales *linearly with batch size* and *quadratically with input resolution*, and it dominates everything else. At batch 32, 224×224, the four models need 478 MB (Simple CNN), 1.91 GB (VGG16), 4.59 GB (ResNet50) and 3.14 GB (EfficientNetB0).
2. **Parameters** — 4 bytes each: ≈ 49 MB, 56 MB, 91 MB and 16 MB respectively.
3. **Gradients** — one float per *trainable* parameter. Frozen backbones make this negligible for the transfer models (0.3–1 MB) but large for the Simple CNN (49 MB).
4. **Optimizer state** — Adam keeps two moment tensors per trainable parameter, so 2× the gradient term.

Summing across all four gives 10.24 GB; adding ~20 % for framework overhead, kernel workspaces and fragmentation gives **≈ 12.3 GB to hold all four training graphs simultaneously at batch 32** — comfortable on this 24 GB machine, impossible on a 16 GB accelerator, and requiring batch 8 (3.4 GB) on an 8 GB card. The measured figures are tabulated in the evidence cell below; the point of the decomposition is that it lets the requirement be *predicted* before an OOM crash rather than discovered by one.

Note how badly a parameter-count intuition would mislead here: ResNet50 has 1.6× VGG16's parameters but needs **2.4× its training memory**, and EfficientNetB0 — the *smallest* model by parameters at 4.2 M — needs more memory than VGG16 at 14.8 M. Depth and activation volume, not weight count, set the budget.

An important qualifier for this specific setup: Apple silicon uses **unified memory**, so the 24 GB pool is shared between host arrays and GPU tensors. The 600 MB of CIFAR arrays and the model activations compete for the same budget — there is no separate "VRAM" to spend.

Responding to an OOM error. Batch size is the correct first lever because activation memory is exactly linear in it and it is the only term that changes without touching the model. Halving 32 → 16 halves the ~5 GB activation term. The practical ladder, in order of how much they cost you:

Step	Memory saved	Cost
Halve batch size (32 → 16 → 8)	linear	noisier gradients, worse GPU utilisation
Gradient accumulation — micro-batch 8, accumulate 4 steps	linear in micro-batch	*no* change in effective batch statistics; extra wall-clock
Mixed precision (float16 activations)	≈ 50 % of activations	small numerical-stability risk
Reduce input 224 → 128	≈ 67 % of activations	accuracy loss on pre-trained backbones (§7.4)
Gradient checkpointing	large	~30 % slower (recomputes activations)
Train models sequentially, not simultaneously	≈ 75 %	wall-clock, not accuracy

Gradient accumulation deserves emphasis: it is the only option that preserves the *effective* batch size, and therefore the gradient-noise scale and the learning-rate/batch-size coupling that §7.1 measures. Simply shrinking the batch changes the optimisation problem; accumulating does not. Note also that if the batch is reduced, the learning rate should generally be scaled with it (the linear-scaling heuristic) — otherwise two variables have changed at once.

```
All four models simultaneously at batch 32 : 10.24 GB
  + 20% framework / workspace overhead      : 12.29 GB  <-- practical estimate
All four at batch 16                        : 6.36 GB
All four at batch 8                         : 3.39 GB
```

Q2 — Data Loading and Preprocessing > *Explain the purpose of setting `validation_split=0.2` in the `ImageDataGenerator`. How > does this affect the training process, and what would happen if you used the same data > generator for both training and validation without this split?*

Q3 — Model Architecture Implementation > *When implementing the transfer learning models, why are the base model layers frozen > initially? What computational and learning advantages does this provide compared to > training all layers from scratch?*

Computational advantages.

* **No backward pass through the backbone.** Backpropagation costs roughly twice the forward pass. With `base.trainable = False` gradients stop at the head, so a training step costs ≈ 1× forward + a trivial head backward instead of ≈ 3× forward. Measured here: VGG16 trains at 156 img/s frozen — training its 14.7M parameters end-to-end would be several times slower. * **No optimizer state for 14–23 M parameters.** Adam's two moment tensors per trainable weight are simply not allocated. For ResNet50 that is ≈ 180 MB of memory not spent. * **Only 0.4–3.9 % of parameters receive gradients** (67K of 14.8M = 0.45 % for VGG16; 264K of 23.9M = 1.1 % for ResNet50; 165K of 4.2M = 3.9 % for EfficientNetB0), so gradient and optimizer memory become rounding errors — visible in the §3.2 memory decomposition, where the transfer models' gradient bars are invisible next to the Simple CNN's.

Learning advantages — the more important half.

* **Sample efficiency.** ResNet50 has 23.6M parameters and this task supplies 40,000 training images: roughly 590 parameters per example. Fitting that end-to-end without massive

Q2 — Data Loading and Preprocessing > *Explain the purpose of setting `validation_split=0.2` in the `ImageDataGenerator`. How > does this affect the training process, and what would happen if you used the same data > generator for both training and validation without this split?*

regularisation would overfit catastrophically. Freezing reduces the effective hypothesis space to a 264K-parameter head — about 6 parameters per example — which is a well-posed estimation problem. * **Preservation of transferable structure.** ImageNet features are hierarchically generic: early layers learn Gabor-like edge and colour-opponent filters that are near-universal for natural images, middle layers learn textures and motifs, and only the last layers become ImageNet-category-specific. Freezing keeps that hierarchy intact instead of destroying it with large early gradients. * **Avoiding catastrophic forgetting.** At initialisation the head is random, so its gradients are large and essentially uninformative. If they propagated into the backbone during the first epochs, they would corrupt pre-trained weights before the head had learned anything useful — the transfer advantage would be destroyed in the first few hundred steps. This is precisely why the standard recipe is *freeze → train head → unfreeze late layers at a 100× lower learning rate*, which is exactly the §8.1 fine-tuning protocol. * **Optimisation stability.** A frozen backbone makes the loss surface far better behaved: the model is effectively a (deep, fixed) feature map followed by a shallow classifier, so training is closer to convex and converges in a handful of epochs.

The trade-off, stated honestly. Frozen ImageNet features are not optimal for upsampled 32×32 CIFAR images — the domains differ in resolution, object scale and image statistics. The ceiling of a frozen backbone is therefore

Q2 — Data Loading and Preprocessing > *Explain the purpose of setting `validation_split=0.2` in the `ImageDataGenerator`. How > does this affect the training process, and what would happen if you used the same data > generator for both training and validation without this split?*

lower than a well-executed full fine-tune. §8.1 quantifies exactly what unfreezing the last stage buys, and §7.4 shows why the resolution mismatch matters.

Q4 — Training Process Management

> *Describe the role of each callback (`EarlyStopping` and `ReduceLROnPlateau`) in the training process. Under what conditions would each callback trigger, and how do they prevent overfitting?*

`EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True)`

Role. It converts "how many epochs?" from a guessed hyper-parameter into a decision the data makes. Training halts once the model stops generalising better, and the weights returned are the best-validated ones rather than the last ones.

Trigger condition. Keras records the best `val_accuracy` seen so far. Every epoch that fails to beat it increments a counter; five consecutive such epochs stop training. A single improvement — however small — resets the counter to zero. Note the interaction with the `patience=5` setting: because a 15-epoch budget is only three patience-windows long, early stopping here mostly guards against wasted compute rather than dominating the schedule.

How it prevents overfitting. Overfitting is not a property of an architecture but of an *optimisation trajectory*: validation performance improves, peaks, then degrades as the model begins fitting noise. Early stopping is a form of **implicit capacity control** — it caps the number of optimisation steps and therefore how far the weights can travel from their initialisation into the high-variance region. `restore_best_weights=True` is what makes it a correctness control rather than merely a time-saver: without it the model returned would be the degraded one from the final epoch.

`ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6)`

Role. It adapts the step size to the curvature the optimiser is currently in. Large steps early cover ground quickly; smaller steps later allow settling into a minimum instead of bouncing across it.

Trigger condition. Three consecutive epochs without an improvement in `val_loss` multiply the learning rate by 0.2 (1e-3 → 2e-4 → 4e-5 → 8e-6 → 1.6e-6 → floor). A plateau in validation loss means the optimiser is oscillating within a basin rather than descending — the exact situation a smaller step resolves.

How it prevents overfitting. Less directly than early stopping, through two mechanisms. First, a smaller learning rate reduces the *effective* capacity explored per epoch: the model makes smaller weight excursions, biasing it toward flatter minima that generalise better. Second, it buys genuine improvement out of epochs that would otherwise be wasted, so the model reaches its best validation point before the early-stopping counter expires.

Why they monitor different quantities — a deliberate design choice.

Validation *loss* degrades earlier than validation *accuracy*, because it is sensitive to growing over-confidence on examples that are still classified correctly. Scheduling the learning rate on the earlier, more sensitive signal while stopping on the metric we actually care about means the scheduler always gets a chance to rescue a plateau before training is abandoned. Because `patience` is 3 for the scheduler and 5 for early stopping, that ordering is guaranteed by construction rather than left to chance.

This is not hypothetical in these runs. The Simple CNN began diverging at epoch 3 (training loss rising past 2.8, validation accuracy falling). `ReduceLROnPlateau` fired at epoch 4, dropped the learning rate to 2e-4, and the run recovered immediately. Had only `EarlyStopping` been configured, the run would have been abandoned at a far worse operating point. The learning-rate trajectories in §4 and the training log show this directly.

```
'weights restored gain' = validation accuracy recovered by restore_best_weights=True
relative to keeping the final-epoch weights. A positive value is overfitting that
EarlyStopping actively undid.
```

Q5 — Memory Management and Efficiency > *Compare the memory footprint during training for the Simple CNN versus the ResNet50 > model. Which operations in each architecture contribute most to memory usage, and how does > this affect training speed?*

Simple CNN fixed 197.4 MB | per batch-unit 14.94 MB | batch 1 ~ 212.4 MB | batch 32 ~ 675.6 MB ResNet50 fixed 94.0 MB | per batch-unit 143.33 MB | batch 1 ~ 237.3 MB | batch 32 ~ 4680.7 MB

The Simple CNN pays up front (trainable weights + Adam state); ResNet50 pays per sample (activations). They nearly converge at batch 1 and diverge ~7x by batch 32.

```
---
### Q6 — Feature Extraction Implementation
> *Explain why different layer extraction strategies are used for each model type (e.g.
> `block5_pool` for VGG16, `last_conv` for Simple CNN). What makes these layers appropriate
> choices for feature representation?*

**The strategies differ in *name* but are identical in *principle*.** Each architecture
places its final convolutional representation at a differently-named tensor, so the layer
identifier must be architecture-specific — but the selection rule is the same for all four:
**take the last tensor that is still spatial and still convolutional, immediately before
global pooling and classification.**

| Model | Layer | What it is |
|-------|-------|-----------|
| Simple CNN | `conv3_128` | output of the third and final `Conv2D` — the deepest learned representation before `Flatten` |
| VGG16 | `block5_pool` | output of the fifth and final convolutional block; the canonical "VGG features" tap used throughout the
literature |
| ResNet50 | `conv5_block3_out` | output of the last bottleneck of stage 5, after the residual addition — the tensor GAP normally
consumes |
| EfficientNetB0 | `top_activation` | the final Swish activation after the 1×1 expansion — EfficientNet's designated embedding point |

**Why this depth is the right choice.** CNNs build a strict feature hierarchy: early layers
respond to edges, colour opponency and orientation (§6.3 shows the Simple CNN's learned
first-layer filters are exactly Gabor-like edge and colour detectors); intermediate layers
compose these into textures and motifs; the deepest convolutional layers encode object parts
and semantic configuration. For a *classification* feature, only the deepest level carries
class-discriminative content — the layer the classifier itself consumes. Extracting from a
mid-network layer would compare a generic texture code in one model against a semantic code
in another, and the comparison would be meaningless.

**Why each specific tap is the correct one for its architecture.**

* `block5_pool` is where VGG16's original classifier attached, so it is the point the network
  was *optimised* to make linearly separable.
* For ResNet50 the tap must be taken **after the residual addition** (`..._out`), not after
  the internal `conv3` of the block — the skip connection is part of the representation, and
  taking the branch output alone would discard the identity path's contribution.
* EfficientNet's `top_activation` follows the final 1×1 expansion that widens the
  representation to 1,280 channels specifically to serve as the embedding; tapping the last
  MBConv block instead would miss that widening.
* The Simple CNN has no pooled equivalent, so the last `Conv2D` output is used directly.

**A consequence worth noting.** The four taps have very different shapes — 56×56×128
(401,408 values), 7×7×512 (25,088), 7×7×2048 (100,352) and 7×7×1280 (62,720). The Simple CNN's
map is *largest in raw dimensionality yet weakest in discriminative power*, because its
spatial resolution has been reduced only three times and its channels encode low-level
structure. Dimensionality is not information. This is why the analysis reports **globally
average-pooled** descriptors for t-SNE and the linear probe: pooling normalises away the
spatial-resolution difference and compares what each model actually encodes per channel.
```

Correlation between raw feature dimensionality and linear-probe accuracy: -0.917 Near-zero or negative → more dimensions do not mean better features.

```
---
### Q7 — Batch Processing and Prediction
> *During the evaluation phase, why is `test_gen.reset()` called before each model
> prediction? What would happen if you forgot this step when evaluating multiple models
> sequentially?*

**What `reset()` does.** A Keras generator is a **stateful iterator**. It holds an internal
batch pointer (`batch_index`) and an index array describing the order in which samples are
emitted; consuming batches advances that pointer. `reset()` rewinds it to zero so the next
pass starts at sample 0 in the canonical order.

**Why alignment matters.** Evaluation compares `y_pred[i]` against `y_true[i]` — a purely
**positional** join. It is valid only if predictions emerge in the same order as the labels.
If a previous operation left the pointer mid-stream, the first prediction returned corresponds
to sample *k*, not sample 0, and every subsequent pairing is shifted.

**The failure is silent, and that is what makes it dangerous.** Nothing raises. With 10
balanced classes a misaligned comparison collapses toward **≈ 10 % — chance level** — because
the pairing has become essentially random. The confusion matrix goes near-uniform and every
per-class F1 falls to ≈ 0.1. When several models are evaluated in sequence, model 1 looks fine
and every model after it looks broken, which naturally leads to blaming the *architecture*
rather than the iterator. A researcher could easily conclude "ResNet50 doesn't work on
CIFAR-10" from what is purely a bookkeeping bug.

**What the measurement below actually shows — an important nuance.** I tested three
scenarios rather than asserting the textbook answer, and the result is more interesting than
the question implies:

| Scenario | With `reset()` | Without |
|---|---|---|
| A. `model.predict(gen)` after advancing the pointer, `shuffle=False` | 0.5533 | **0.5533** |
| B. Manual `next(gen)` consumption after advancing (Keras 2's `predict_generator` pattern) | 0.5533 | **0.1073** |
| C. `shuffle=True` on the test generator | — | **0.0965** |

In **Keras 3, `model.predict()` re-enumerates the iterator from the beginning**, so with
`shuffle=False` the reset turns out to be *defensive rather than load-bearing in this
version* (scenario A). The corruption is entirely real in the two scenarios that matter:

* **Scenario B** — any hand-rolled evaluation loop, and the `predict_generator` API this
  pattern comes from, consumes the iterator *in place*. Accuracy collapses from 0.5533 to
  **0.1073**, essentially chance.
* **Scenario C** — with `shuffle=True` the generator emits a permuted order, so `reset()`
  alone cannot save the join. Accuracy is **0.0965**.

The honest conclusion: `reset()` is correct and necessary practice, but the reason it is safe
to omit in this particular code path is an *implementation detail of the framework version*,
not a guarantee of the API contract. Relying on it would be a latent bug — one library upgrade
away from silently producing chance-level results.

**Additional safeguards used here.**

* `shuffle=False` on the test generator, so the order is deterministic in the first place.
  Scenario C shows this is the more fundamental of the two controls: `reset()` cannot rescue a
  shuffled stream.
* No augmentation on the test generator, so the images are the actual test images.
* `evaluate_model` calls `test_gen.reset()` as its **first statement**, making alignment a
  structural property of the function rather than something the caller must remember.

The most robust design avoids the shared mutable state entirely — construct a fresh generator
per model, or predict directly from the NumPy array. `reset()` is the right fix when a single
generator object is reused across models, which is the pattern the study protocol describes.
```

Random-guess baseline for 10 balanced classes: 0.1000

A: Keras 3's predict() re-enumerates the iterator from index 0, so with shuffle=False the reset is DEFENSIVE, not load-bearing – in this framework version. B: a hand-rolled loop consumes the iterator in place: 0.5690 → 0.1064, a collapse to chance level. This is the real, silent failure mode. C: with shuffle=True the emitted order is permuted and reset() cannot help: 0.0965.

Conclusion: reset() is correct practice; the fact that scenario A survives without it is an implementation detail of this Keras version, not a guarantee of the API contract.

```
---
### Q8 — Data Augmentation Impact
> *The training data generator includes rotation, shifting and flipping augmentations. How
> do these augmentations affect the training time and convergence behaviour compared to
> training without augmentation?*

**Effect on training time — two separable costs.**

1. **Per-epoch wall-clock.** Each transform is an affine warp computed on the CPU. Measured
   here, the augmented generator sustains roughly 9,000 images/s at 32×32 (the evidence cell
   below re-measures it live), while the GPU consumes only 156–299 images/s. The augmentation pipeline is therefore ~30× faster than
the
   consumer and is **fully hidden behind GPU compute** — per-epoch time is essentially
   unchanged. This would *not* hold if augmentation were applied after upsampling to
   224×224 (49× more pixels per warp), which is one more reason the pipeline augments at
   native resolution and upsamples in-graph.
2. **Epochs to convergence.** This is where augmentation genuinely costs time. Every epoch
   presents *different* images, so the effective training distribution is larger and noisier.
   Training accuracy rises more slowly and to a lower ceiling, and more epochs are needed to
   reach the same training-set fit. Total time-to-target-accuracy typically increases even
   though time-per-epoch does not.

**Effect on convergence behaviour.**

* **The train/validation gap shrinks, and can invert.** Without augmentation, training
  accuracy races ahead of validation accuracy — the signature of memorisation. With
  augmentation, the model is evaluated on clean validation images but trained on distorted
  ones, so validation accuracy frequently *exceeds* training accuracy. A negative gap is not
  an error; it is evidence the regulariser is doing its job.
* **Noisier epoch-to-epoch curves.** Each epoch samples a different realisation of the
  augmented distribution, so both loss and accuracy fluctuate more. This interacts with the
  callbacks: a noisy `val_loss` can trip `ReduceLROnPlateau` on what is really sampling
  noise, which is an argument for the patience values used rather than tighter ones.
* **A better final generalisation point.** The model is forced to learn representations
  invariant to rotation, translation and reflection instead of exploiting position- or
  orientation-specific shortcuts.
* **Delayed overfitting onset.** The epoch at which validation loss turns upward is pushed
  later, which is what allows a longer effective training schedule.

**Why *these* transforms.** Each encodes a genuine invariance of the data-generating process:
±15° rotation ↔ camera roll; ±10 % shift ↔ imperfect centring (the padding-and-crop surrogate
the study protocol asks for); horizontal flip ↔ left–right symmetry, which holds for all ten
CIFAR classes. **Vertical flipping is deliberately excluded** — an upside-down ship is not a
plausible sample, so it would inject label-preserving noise the network must spend capacity
ignoring. Augmentation is only useful when it expands the data distribution along directions
the *test* distribution actually varies in.

**Caveat specific to frozen transfer models.** Augmentation regularises only the trainable
parameters. For VGG16/ResNet50/EfficientNetB0 the trainable set is a 67K–264K head, which has
limited capacity to overfit 40,000 images in the first place, so augmentation's marginal
benefit is much smaller there than for the 12.9M-parameter Simple CNN. Its cost is also
non-zero — it makes the frozen backbone's features noisier. This is visible in the §5.1
generalisation-gap panel, where the transfer models sit close to zero throughout.
```

augmented : 7721 images/s (CPU generator) no augmentation : 151679 images/s (CPU generator) GPU consumption : 156 - 299 images/s ← the actual bottleneck

The augmentation pipeline is ~30x faster than the GPU can consume it, so it is fully overlapped and costs ~0 additional wall-clock per epoch.

```
---
### Q9 — Model Compilation and Optimization
> *All models use the Adam optimizer with categorical crossentropy loss. Justify this choice
> for the multiclass classification task, and explain when you might consider using different
> optimizers or loss functions.*

**Categorical cross-entropy is the correct loss.** The task is single-label classification
over 10 mutually exclusive classes with one-hot targets. Minimising
$\mathcal{L} = -\sum_c y_c\log\hat{y}_c$ is exactly maximum-likelihood estimation under a
categorical model, so the loss is not a heuristic — it *is* the statistical model. Three
practical consequences:

* **Non-saturating gradients.** With a softmax output the gradient at the logits is
  $\hat{y}-y$: linear in the error, and largest precisely when the model is confidently
  wrong. MSE on a softmax output has a gradient proportional to $\sigma'$, which *vanishes*
  in exactly that regime — the model gets almost no signal when it most needs it.
* **Calibrated probabilities.** Cross-entropy is a proper scoring rule, so the softmax
  outputs are meaningful confidences. The reliability diagram in §5.4 and the soft-voting
  ensemble in §8.3 both depend on this; a hinge loss would give a decision boundary but no
  usable probabilities.
* **Correct treatment of exclusivity.** The softmax couples the classes so they sum to one,
  encoding the fact that an image is one class, not several.

**Adam is the right default here.** Adam maintains per-parameter first and second moment
estimates, giving each weight an effective step size scaled by its own gradient history.
Three properties of *this* problem make that valuable:

1. **Heterogeneous gradient scales within one model.** A freshly-initialised `Dense(10)` head
   sits on top of a frozen backbone; a single global step size would be simultaneously too
   large for one part and too small for another.
2. **Sparse ReLU gradients.** §6 measures 83–97 % activation sparsity — most units receive
   gradient only intermittently. Adam's accumulated second moment keeps those units learning;
   plain SGD effectively stalls them.
3. **Robust defaults under a one-run budget.** $\beta_1{=}0.9$, $\beta_2{=}0.999$ work across
   all four architectures without per-model tuning, which matters when the compute budget
   allows one run per configuration rather than a sweep.

**When to choose differently.**

| Situation | Better choice | Why |
|-----------|---------------|-----|
| Long from-scratch training, chasing the last 1 % | **SGD + Nesterov momentum + cosine schedule** | converges to flatter minima; the
original ResNet results used it. Needs careful LR tuning |
| Transformers, or any model needing weight decay | **AdamW** | decouples weight decay from the adaptive step, which plain Adam
corrupts |
| Recurrent networks | **RMSprop** | Adam without bias-correction overhead; stable on RNN gradients |
| Very large batches (≥ 4,096) | **LARS / LAMB** | layer-wise adaptive rates keep large-batch training stable |
| Severe class imbalance | **focal loss**, or class-weighted CE | down-weights easy majority examples |
| Multi-**label** (an image can be several classes) | **binary cross-entropy** + sigmoid | classes are no longer mutually exclusive |
| Noisy labels / over-confidence | **label smoothing** (ε ≈ 0.1) | targets 0.9/0.011 instead of 1/0; improves calibration |
| Metric learning / retrieval | **triplet or contrastive loss** | optimises embedding geometry, not class posteriors |
| Memory-constrained, integer labels | `sparse_categorical_crossentropy` | identical mathematics, avoids materialising one-hot targets
|

**For CIFAR-10 specifically**, the dataset is exactly balanced (§2.1), single-label and
clean-labelled, so none of the specialised alternatives is warranted. Plain categorical
cross-entropy with Adam is not a lazy default — it is the choice the data justifies.
And empirically, §7.2 shows the accuracy span between a *well*- and a *badly*-chosen learning
rate is far larger than any plausible Adam-vs-SGD difference: getting the step size right
matters more than the optimizer brand.

---
### Q10 — Reproducibility and Random Seeds
> *The notebook sets random seeds for NumPy and TensorFlow. Identify all the sources of
> randomness in the training pipeline and explain why setting these seeds is crucial for
> reproducible experiments.*

**Every source of randomness in this pipeline, and how each is controlled.**

| # | Source | Where it acts | Controlled by |
|---|--------|---------------|---------------|
| 1 | Weight initialisation (Glorot uniform, zeros for bias) | every `Conv2D` / `Dense` | `tf.random.set_seed` |
| 2 | Train/validation partition | `prepare_arrays` shuffle before the index cut | `np.random.RandomState(SEED)` |
| 3 | Batch shuffling order each epoch | `flow(shuffle=True, seed=...)` | generator `seed` argument |
| 4 | Augmentation transform sampling (angle, shift, flip, zoom) | `ImageDataGenerator` | Python `random` + generator `seed` |
| 5 | Dropout masks | `Dropout(0.5)` at every training step | `tf.random.set_seed` |
| 6 | Feature-analysis subset selection | 200 images/class draw | `np.random.RandomState(SEED)` |
| 7 | t-SNE initialisation and gradient-descent noise | `sklearn.manifold.TSNE` | `random_state=SEED` |
| 8 | PCA sign/rotation ambiguity, k-fold assignment | `PCA`, `StratifiedKFold` | `random_state=SEED` |
| 9 | Python container hash ordering | dict/set iteration | `PYTHONHASHSEED` |
| 10 | **Non-deterministic GPU kernel reductions** | `cuDNN`/Metal atomics, autotuning | `TF_DETERMINISTIC_OPS` — *partially* |

`set_seeds()` covers 1–9. Item 10 is the honest limitation and is discussed below.

**Why this matters — four distinct reasons.**

1. **Attribution of causes.** The §7 ablations claim that changing the learning rate changes
   accuracy. That claim is only valid if *nothing else* changed. Without seeding, the observed
   difference could be initialisation luck. Every ablation calls `set_seeds()` immediately
   before building its model, so all variants start from identical weights and see identical
   batch orders — the modified factor is genuinely the only difference.
2. **Debugging.** A non-deterministic pipeline makes intermittent bugs nearly impossible to
   isolate, because a fix cannot be distinguished from a lucky run.
3. **Verification by others.** The §5 numbers are only checkable if a third party can
```

```
   regenerate them. Seeding plus the pinned `environment_report()` versions is what makes the
   deliverables auditable rather than merely asserted.
4. **Honest uncertainty.** Seeding fixes *one* sample from the distribution over training
   runs — it does not tell you how wide that distribution is. That is a separate question,
   answered by the cross-validation in §8.2, whose standard deviation gives the scale against
   which the model-to-model differences in §5.2 should be judged. This is why §5.5 also
   applies McNemar's test rather than simply comparing point estimates.

**The limits of seeding — stated honestly.** Bit-exact reproducibility across *different*
hardware or library versions is not achievable and is not claimed here:

* GPU floating-point reductions are order-dependent; parallel atomics change the summation
  order run to run. `TF_DETERMINISTIC_OPS=1` constrains this but does not cover every kernel,
  and coverage on the Metal PluggableDevice backend differs from CUDA.
* cuDNN/Metal algorithm autotuning may select different kernels depending on transient
  machine state.
* Floating-point addition is non-associative, so even mathematically identical computations
  can differ in the last bits, and those differences amplify over thousands of steps.

The defensible claim is therefore: **on this machine with these pinned library versions, runs
are reproducible; across environments, results are reproducible to within the run-to-run
variance quantified in §8.2** — not bit-identical. Everything needed to check this is
recorded: the seed, the full version manifest, the per-epoch CSV logs, and the exported
JSON/CSV results.
```

def set_seeds(seed: int = SEED) → None: “””Seed every source of randomness in the pipeline.

```
Sources covered:
  * Python `random`      -> ImageDataGenerator transform sampling
  * NumPy                -> array shuffling, train/val partition, t-SNE init
  * TensorFlow           -> weight initialisation, dropout masks, shuffling
  * PYTHONHASHSEED       -> hash-ordering of Python containers
"""
random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
tf.keras.utils.set_random_seed(seed)
```

```
```

Identical conv1 initialisation across two seeded builds : True (max |diff| = 0.00e+00) Identical first augmented batch across two seeded runs : True (max |diff| = 0.00e+00)

Environment manifest pinned with the results: python_version : 3.9.6 platform : Darwin 25.5.0 (arm64) processor : arm tensorflow_version : 2.16.2 keras_version : 3.10.0 numpy_version : 1.26.4 pandas_version : 2.3.3 gpu_devices : [‘/physical_device:GPU:0’] gpu_available : True num_cpus : 10 seed : 42 mixed_precision_policy : float32

Run-to-run variance (from the 3-fold CV in section 8.2): validation accuracy SD = 0.0084 → differences between models smaller than ~0.0167 should not be treated as meaningful without a significance test.

9.2 Parameter Modification Questions

Each answer states the mechanism, then reads the measured result out of the §7 ablation tables. Recall the caveat established in §7: these runs use a fixed 8,000/2,000 subset and a 6-epoch budget, so **rankings and trends are the finding, not absolute accuracies**.

PM1 — Batch Size (16 vs 32 vs 64)
> *How do these changes affect training time, memory usage and final model accuracy? Explain
> the trade-offs and determine the optimal batch size for your hardware.*

Mechanism.

* *Memory* scales **exactly linearly** with batch size — the activation term dominates, and every layer output is stored `batch_size` times. This is the most predictable of the three effects.
* *Time per epoch* does **not** scale linearly, and the direction depends on where the bottleneck is. Larger batches mean fewer, bigger kernel launches, so GPU utilisation and arithmetic intensity improve and per-epoch time usually *falls* — until the batch is large enough to saturate the device, after which the curve flattens. Small batches suffer from per-step overhead (kernel launch, Python-side generator dispatch) that is amortised poorly.
* *Accuracy* is affected through **gradient noise**. The gradient-noise scale is ∝ 1/√batch_size, so small batches produce noisy updates. That noise acts as an implicit regulariser, which is often *beneficial* for generalisation — the well-documented "large-batch generalisation gap". Small batches also perform more parameter updates per epoch (8,000/16 = 500 steps vs 8,000/64 = 125), so under a **fixed epoch budget** the small batch simply gets 4× more optimisation steps. That confound is real and is flagged in the reading of the results below.

Trade-offs. Batch size sets a three-way exchange between memory (linear), throughput (sub-linear, saturating) and gradient quality (√-law). It is also coupled to the learning rate — the linear-scaling heuristic says doubling the batch should roughly double the LR to keep the effective step size constant. Since the LR is held fixed at 1e-3 here, part of any accuracy difference is attributable to that coupling rather than to batch size alone.

Measured result.

batch	test accuracy	weighted F1	s/epoch	steps/epoch	activation MB	est. train MB
16	0.4135	0.3994	50.2	500	239	437
32	0.4514	0.4233	35.2	250	478	676
64	0.4524	0.4418	32.7	125	956	1154

* **Memory** behaves exactly as predicted: 239 MB at batch 16 → 956 MB at batch 64, a factor of 4.0× for a 4× batch increase — linear to within rounding.
* **Fastest epoch:** batch 64 at 32.7 s, versus 50.2 s for the slowest — a 53% spread.
* **Most accurate:** batch 64 at 0.4524, spanning 0.0389 across the three settings.

Optimal batch size for this hardware: 64 — it gives the best accuracy at 32.7 s/epoch and 1154 MB, comfortably inside the 24 GB unified-memory budget. Batch 32 (the study protocol default) is the pragmatic choice for the headline runs: it sits within 0.0010 of the best accuracy while leaving enough memory headroom to train the 224×224 transfer models. If an OOM forced a reduction, batch 16 costs 0.0379 accuracy — a price worth paying over not training at all, and recoverable via gradient accumulation.

PM2 — Learning Rate (0.0001 vs 0.001 vs 0.01)
> *How do the training curves differ, and which learning rate achieves the best validation
> accuracy? What signs of too-high or too-low learning rates do you observe?*

Mechanism. The learning rate sets the step size in a loss landscape whose curvature it knows nothing about. Too large and the step overshoots the basin, so the loss increases or oscillates — in the worst case the weights grow without bound and activations saturate. Too small and each step makes negligible progress; the model is not *wrong*, just **budget-starved**, and under a fixed epoch count it simply never arrives. The response curve is therefore an inverted U, and its peak is the largest step size that remains stable.

Diagnostic signatures to look for.

Too high (≈ 1e-2 here):
* training loss **increases** or plateaus at a high value from the first epochs;
* violent epoch-to-epoch oscillation in both loss and accuracy;
* accuracy stuck near chance (0.10 for 10 balanced classes) — a saturated/collapsed model;
* in the worst case, `NaN` loss from exploding activations.

Too low (≈ 1e-4 here):
* both curves are **smooth and monotone but shallow** — the shape is healthy, the slope is not;
* training and validation accuracy stay close together (no overfitting, because the model has not yet fit anything);
* loss still falling steadily when the epoch budget expires — the tell-tale sign that the run was truncated rather than converged.

Well-chosen: rapid early descent, then a flattening curve, with the train/validation gap

beginning to open — evidence the model has reached the capacity-limited regime rather than the step-size-limited one.

Which rate is "well-chosen" is a property of the model, not a universal constant, and this architecture is a case in point. The Simple CNN's 12.9 M-parameter `Flatten → Dense(128)` head makes it unusually sensitive to step size: the headline run in §4 was trained at the Adam default of 1e-3 and **began diverging at epoch 3** (training loss climbing past 2.8), recovering only after `ReduceLROnPlateau` cut the rate to 2e-4. If that diagnosis is right, the ablation should independently show a rate *below* the default outperforming it. The measured table below is therefore a falsifiable prediction, not a formality.

Measured result.

learning rate	best val acc	test accuracy	weighted F1	final train acc	final val acc	train−val gap
1e-04	0.4935	0.4903	0.4683	0.3765	0.4935	-0.1170
1e-03	0.4785	0.4514	0.4233	0.3736	0.4570	-0.0834
1e-02	0.1040	0.1000	0.0182	0.0988	0.1015	-0.0027

Best validation accuracy: lr = 1e-04 (0.4935).

* **lr = 1e-02 — diverged — step size exceeds the stable range.** Test accuracy 0.1000 against a 0.10 chance baseline: the model has collapsed to predicting essentially one class. The §7.2 log-scale loss plot shows the loss failing to descend at all — the optimiser overshoots every basin it enters.
* **lr = 1e-03 (the Adam default) — stable but sub-optimal.** Test accuracy 0.4514.
* **lr = 1e-04 — best of the three.** Test accuracy 0.4903.

The prediction stated above is confirmed. The best rate is **1e-04**, which is below the Adam default of 1e-3 — the ablation independently reproduces the instability seen in the headline run, where this same architecture diverged at 1e-3 and recovered only once `ReduceLROnPlateau` cut the rate to 2e-4. Two experiments run on different data subsets, different epoch budgets and different schedules agree on the same diagnosis: **the 12.9 M-parameter `Flatten → Dense` head cannot tolerate the default step size.** The lesson is not "1e-4 is better than 1e-3" in general — it is that the optimal step size is a property of the architecture, and this architecture advertises the problem.

The accuracy span across a single order of magnitude in either direction is **0.3903** — far larger than the span between *any two architectures* in the headline comparison (0.3523). That is the practical lesson: learning rate is the highest-leverage hyper-parameter in the pipeline, which is precisely why `ReduceLROnPlateau` earns its place in the callback set.

PM3 — Dropout Rate (0.5 → 0.3 → 0.8)

> *How does this change affect overfitting behaviour and final test accuracy? Which dropout rate provides the best balance between underfitting and overfitting?*

Mechanism. Dropout zeroes each unit independently with probability *p* at training time, scaling the survivors by 1/(1−p) to preserve expected magnitude. Two effects follow: units cannot co-adapt (each must be independently useful, since its neighbours may vanish), and training approximates an ensemble over 2^n thinned sub-networks. At inference all units are active, so the prediction is an implicit ensemble average.

p is therefore a regularisation dial with a genuine optimum:
* **too low** → co-adaptation persists, the head memorises, the train−val gap widens;
* **too high** → so much signal is destroyed per step that the layer cannot fit the data at all. Gradients become extremely noisy and convergence slows sharply. At p = 0.8 only ~26 of the 128 head units survive any given step.

The context matters more than the number. In these transfer models dropout regularises **only the 128-unit head** — the frozen backbone cannot overfit, because it cannot change. The trainable capacity is 165K parameters against 8,000 training images, which is a mild overfitting risk to begin with. Heavy dropout is therefore expected to hurt (underfitting) more than light dropout hurts (overfitting) — the opposite of what one would predict for the 12.9M-parameter Simple CNN, where the `Flatten → Dense` head is a genuine overfitting engine.

Measured result (EfficientNetB0 head, frozen backbone):

dropout	test accuracy	weighted F1	final train acc	final val acc	train−val gap
0.3	0.8728	0.8726	0.8213	0.8975	-0.0762
0.5	0.8760	0.8757	0.8005	0.8895	-0.0890
0.8	0.8668	0.8663	0.7166	0.8780	-0.1614

Best balance: p = 0.5 (0.8760 test accuracy).

* The **overfitting gap** moves from -0.0762 at p = 0.3 to -0.1614 at p = 0.8 — a change of -0.0851, in the expected direction: heavier dropout suppresses the train-minus-validation gap.
* **At p = 0.8** accuracy is 0.8668, 0.0092 below the best setting. This is the signature of a model regularised past the point of usefulness: the gap is smallest but so is the accuracy. A shrinking gap is only good news while accuracy holds.
* **p = 0.3 vs p = 0.5** differ by 0.0032 — small, because the frozen backbone leaves little to overfit. The study protocol's default of 0.5 is a defensible middle setting; the ablation shows the answer would likely differ for a fully-trainable network, where the overfitting pressure is far higher.

The general lesson: the right dropout rate is a function of the ratio between *trainable*
capacity and dataset size, not a universal constant. Reporting only the gap would have made
p = 0.8 look best; reporting accuracy alongside it shows why that reading is wrong.

PM4 — Image Size (224 → 128 → 384)
> *How do these modifications affect training time, model performance and memory
> requirements? Consider both the transfer-learning models (pre-trained on 224×224) and the
> simple CNN.*

Mechanism. Compute and activation memory scale with **pixel count**, i.e. quadratically
with the side length: $128^2 / 224^2 \approx 0.33\times$ and $384^2 / 224^2 \approx 2.94\times$. For a `Flatten`-headed
network the *parameter* count also scales quadratically, because the flattened vector is
$(\text{size}/8)^2 \times 128$ units.

The two model families respond differently, and for different reasons.

Transfer models were pre-trained at 224×224. Their learned filters have a receptive-field
geometry matched to that scale, and their fixed downsampling factor (32× for VGG16 and
ResNet50) means input size determines the final feature-map resolution: 224 → 7×7,
128 → 4×4, 384 → 12×12. The textbook expectation is that deviating from 224 puts objects at a
scale the filters were never fitted to, so accuracy should fall. **This expectation is worth
testing rather than assuming** — the source images are 32×32 either way, so the "native
resolution" argument may matter far less here than it would for genuine 224×224 photographs.

The Simple CNN has no pre-training to violate, so it is free to use whatever resolution is
given — but its information content is capped regardless: **the source images are 32×32.**
Upsampling to 384 adds pixels, not evidence. The 3,072 original numbers are all the model
ever has. Larger inputs therefore buy nothing but cost quadratically, while inflating the
`Flatten → Dense` head to tens of millions of parameters.

The prediction, before looking: 384 should cost ~3× the compute of 224 for no accuracy
gain in either family; 128 should be ~3× cheaper, hurting the pre-trained model more than the
from-scratch one.

Measured result — Simple CNN:

input	params	test accuracy	s/epoch	rel. time vs 224	rel. pixels	est. train MB
128^2	4,288,970	0.4946	14.8	0.42×	0.33×	222
224^2	12,939,722	0.4514	35.5	1.00×	1.00×	676
384^2	37,843,402	0.4322	101.5	2.86×	2.94×	1982

Measured result — VGG16 (pre-trained at 224×224):

input	test accuracy	s/epoch	final feature map
128^2	0.8014	39.5	4×4×512
224^2	0.7924	112.7	7×7×512

* **Time tracks pixel count closely**, confirming the quadratic law: 384^2 costs
  2.86× the time of 224^2 (predicted
  2.94×) and 128^2 costs 0.42×
  (predicted 0.33×).
* **Parameters explode with the `Flatten` head:** 4,288,970 at 128^2
  → 37,843,402 at 384^2, a
  8.8× increase driven entirely
  by the flattened vector.
* **384^2 costs 2.9× the compute of 224^2
  and returns -0.0192 accuracy**
  (0.4322 vs 0.4514) —
  no gain, exactly as predicted.
  Upsampling cannot create information the 32×32 source never contained; it only enlarges the
  tensors that carry it.
* **VGG16 at 128^2 reaches 0.8014 against
  0.7924 at its native 224^2** — a change of
  +0.0090 for
  2.9× less compute per epoch.
  This contradicts the textbook expectation that a pre-trained backbone must be fed at its training resolution.
  The difference is small — comparable to the 0.0084
  run-to-run standard deviation measured by cross-validation in §8.2 — so the honest reading is
  "no detectable penalty at 128", not "128 is better". The mechanism is that the evidence
  ceiling is set by the 32×32 source, not by the network input: at 128 the final feature map is
  4×4 rather than 7×7, but neither carries more information than the original 3,072 pixels.
  Feeding a pre-trained backbone below its training resolution is usually costly; when the
  images are upsampled thumbnails, that cost largely disappears — and 224 stops being free.

Recommendation. For the transfer models, 224^2 is the *safe* default but not a free one:
the measurement above shows no detectable accuracy penalty at 128^2 for
2.9× less compute per
epoch, precisely because the underlying images are 32×32 upsampled thumbnails rather than true
high-resolution photographs. Going *above* 224 is unambiguously wasteful. For the Simple CNN, 224^2 is a pure waste: it has no pre-
training to respect, so the
smallest resolution that preserves the source information (32^2, the native size) is optimal.
That is precisely what the `Simple_CNN_Native32` baseline in §5.5 demonstrates — the same
architecture, dramatically cheaper, at comparable accuracy.

PM5 — Architecture Depth (adding `Conv2D(256) → MaxPooling2D`)
> *How does this architectural change affect the total parameter count, training time and

```
> classification performance? Compare the feature-extraction capabilities of the original
> versus modified architecture using t-SNE visualisation.*

**Mechanism, and a counter-intuitive prediction.** Adding a fourth block appends a
`Conv2D(256)` (295K new parameters) **and** a fourth `MaxPooling2D`, which halves the spatial
resolution again: 28×28 → 14×14. The `Flatten` vector therefore shrinks from
28×28×128 = 100,352 units to 14×14×256 = 50,176 — **half the size**. Since the
`Flatten → Dense(128)` matrix is 99 % of this network's parameters, halving its input halves
the network.

So the prediction is that **making the network deeper makes it smaller** — a result that is
counter-intuitive only if one assumes depth and parameter count move together. They do not:
in a `Flatten`-headed CNN, parameters live in the transition to the classifier, and every
additional pooling stage cuts that transition quadratically.

Representationally, the extra block adds one more level of composition (larger effective
receptive field, more abstract features) and 256 channels instead of 128 — so the *pooled*
descriptor is richer even though the raw map is smaller.


**Measured result.**

| metric | 3 blocks (baseline) | 4 blocks (+Conv2D 256) | change |
|---|---|---|---|
| Total parameters | 12,939,722 | 6,812,362 | -47.4% |
| Flatten input units | 401,408 | 200,704 | — |
| Test accuracy | 0.4514 | 0.4422 | -0.0092 |
| Weighted F1 | 0.4233 | 0.4154 | -0.0079 |
| Seconds / epoch | 35.7 | 38.5 | +7.9% |
| Inference ms/image | 0.667 | 0.785 | +17.7% |
| Feature sparsity | 97.6% | 97.9% | — |
| Overfit gap | -0.0834 | -0.1004 | — |

* **The parameter count fell** by
  6,127,360 despite adding a layer — the fourth pooling
  stage halves the `Flatten` vector, and that saving
  outweighs the 295K
  parameters of the new convolution. Depth and parameter count are not the same axis.
* **Accuracy change: -0.0092**, and the per-epoch cost
  changed by +7.9% —
  the extra convolution operates on a smaller map, so it is nearly free.
* **t-SNE (§7.5)** compares the two feature spaces directly: the silhouette and 2-D 10-NN
  scores quantify whether the deeper representation actually separates classes better, rather
  than relying on visual impression of the scatter plots.

**Interpretation.** Under this 6-epoch, 8,000-image budget the deeper model is
not better on accuracy while
being smaller and roughly as fast
— so on a pure efficiency basis the four-block variant is the more sensible design. The wider
lesson is that "make it deeper" and "make it bigger" are independent decisions, and for a
`Flatten`-headed architecture they can point in opposite directions. Replacing `Flatten` with
`GlobalAveragePooling2D` — as all three transfer models do — removes the coupling entirely
and would reduce either variant to well under 500K parameters.


---
## 9.3 Trade-off Synthesis

**Objective:** *Discuss trade-offs (e.g. accuracy vs efficiency).*
```

Spearman rank correlation with training wall-clock: total parameters : rho = +0.600 trainable parameters : rho = -0.800 (n = 4 models – descriptive only, not a significance test) → neither weight count explains training cost on its own; forward-pass arithmetic (depth x spatial resolution) is the missing variable.

What the numbers actually say

Accuracy vs training cost. Total parameter count is a poor predictor of training cost. VGG16 carries 14.8 M weights but trains 67 K of them (0.45 %); ResNet50 carries 23.9 M and trains 264 K (1.1 %); EfficientNetB0 carries 4.2 M and trains 165 K (3.9 %). Freezing removes the backward pass through the backbone entirely, so what remains — and what dominates per-epoch cost — is **forward-pass arithmetic**, not weight count.

The decisive evidence is the **VGG16 / ResNet50 inversion**: ResNet50 carries 1.6× VGG16's parameters (23.9 M vs 14.8 M) and updates 4× as many of them (264 K vs 67 K), yet it trains in roughly **two-thirds of VGG16's wall-clock time** — and is the more accurate model. Parameter count does not merely fail to predict training cost here; it predicts the ordering *backwards*.

The reason is architectural. VGG16 stacks wide 3×3 convolutions at high spatial resolution before each downsample, which is enormously arithmetic-heavy. ResNet50 is deeper but spends most of its layers on 1×1 bottleneck projections operating on already-reduced feature maps, so it performs far less arithmetic per forward pass despite holding more weights. Freezing buys a large *constant-factor* saving over fine-tuning the same backbone, but it does not make an arithmetic-heavy backbone cheap in absolute terms — which is why the Simple CNN, with a comparatively light forward pass, remains the fastest of the four to train despite updating every one of its 12.9 M parameters.

Anyone budgeting compute from parameter counts alone will mis-plan in both directions.

Accuracy vs inference cost. This trade-off does *not* invert. At inference every parameter participates in the forward pass, so the large backbones are genuinely slower and larger on disk. A model that is cheap to train can be expensive to serve — training cost and serving cost are separate budgets that happen to be measured in the same units.

Accuracy vs data efficiency. The frozen-backbone models reach their operating point within a few epochs because they are fitting a small head on strong features. The Simple CNN must learn its features from 40,000 images and needs its full budget — and, as §4 showed, was one learning-rate reduction away from not converging at all. Under a *tight* compute budget transfer learning wins on robustness as much as on accuracy.

Capacity vs generalisation. More parameters did not monotonically mean better accuracy — in §7.5 the *deeper* Simple CNN was also the *smaller* one, and in §5.2 the parameter ranking and the accuracy ranking do not match. What matters is whether capacity is placed where it buys discriminative power (convolutional depth) or merely where it burns memory (a `Flatten` transition).

The honest ceiling. Frozen ImageNet features are extracted from images upsampled from 32×32. No amount of head-tuning recovers detail that was never in the source. §8.1 shows what unfreezing the final stage buys, and that is the direction with headroom left — not bigger heads or longer schedules.

9.4 Reproducibility Statement

Requirement	How it is satisfied
Fixed random seeds	`SEED = 42` applied to Python `random`, NumPy, TensorFlow, `PYTHONHASHSEED`; `set_seeds()` is called before *every* model build, including each ablation variant
Deterministic data pipeline	fixed-seed pre-split shuffle; index-based `validation_split`; `shuffle=False` on validation and test iterators
Pinned environment	full version manifest recorded by `environment_report()` and exported to `cnn_lab_results.json`
Single code path	notebook and all four driver scripts import the same `capstone_lib.py` — no possibility of drift between what was trained and what is reported
Raw training records	per-epoch `CSVLogger` output (`artifacts/*_epochs.csv`) and verbatim Keras stdout (`artifacts/*_trainlog.txt`)
Machine-readable results	`cnn_lab_results.json` and `cnn_lab_comparison.csv`
Uncertainty quantified	3-fold cross-validation (§8.2) gives the run-to-run standard deviation; McNemar's test (§5.5) tests model differences against it
Full rebuild	delete `artifacts/` and run the four drivers in order

Stated limitation. Bit-exact reproducibility is *not* claimed across different hardware or library versions. GPU floating-point reductions are order-dependent, `TF_DETERMINISTIC_OPS` does not cover every kernel on the Metal backend, and floating-point addition is non-associative. The defensible claim is reproducibility *on this machine with these pinned versions*, and reproducibility elsewhere to within the variance measured in §8.2.

9.5 Limitations, Ethics and Future Work

Limitations

1. **Resolution ceiling.** Every result is bounded by the 32×32 source. Upsampling to 224 matches the backbones' geometry but adds no information; the frozen ImageNet features are being asked to describe images far blurrier than anything in their training distribution.
2. **Frozen-backbone ceiling.** The headline models tune only 67K–264K head parameters. §8.1 quantifies the headroom left on the table; a full fine-tune with a longer schedule would very likely exceed all four headline numbers.
3. **Reduced-scale ablations.** §7 uses 8,000 training images and 6 epochs. Rankings are controlled and valid; absolute values are not comparable to §5 and must not be quoted as if they were.
4. **Single run per configuration.** Apart from the 3-fold CV, each cell of the study is one run. Differences smaller than roughly twice the CV standard deviation should not be treated as real — which is why §5.5 reports McNemar's test rather than raw deltas.
5. **One dataset, one domain.** CIFAR-10 is balanced, clean-labelled, small-object and centre-cropped. None of these conclusions should be assumed to transfer to imbalanced, noisy, high-resolution or fine-grained problems without re-testing.
6. **Hardware specificity.** Timings come from one Apple-silicon unified-memory GPU via the Metal backend. The *relative* ordering should hold on CUDA hardware; the absolute numbers and the CPU/GPU bottleneck balance will not.

````
### Ethics and governance

* **Provenance and bias.** CIFAR-10 is a curated academic benchmark, but the ImageNet
  backbones carry ImageNet's documented geographic and category biases. Those biases are
  inherited wholesale by a frozen feature extractor, and nothing in this pipeline detects or
  corrects them. On a benchmark this is harmless; in a deployed classifier over
  people-adjacent categories it would not be.
* **Explainability.** The t-SNE projections and feature-map visualisations in §6 give partial
  insight into *what* is encoded, but they are descriptive, not causal. A deployment in a
  regulated setting would need attribution methods (Grad-CAM, integrated gradients) and a
  documented failure taxonomy, not a scatter plot.
* **Calibration and deployment risk.** §5.4 measures expected calibration error precisely
  because a model that is *confidently* wrong is more dangerous than one that is uncertainly
  wrong. Any downstream use should threshold on confidence and route low-confidence cases to
  human review.
* **Environmental cost.** This study consumed several GPU-hours. The efficiency framing in
  §9.3 is not only an engineering concern — reporting accuracy-per-unit-compute alongside
  accuracy is part of responsible reporting.
* **Scope discipline.** These models classify 10 coarse object categories at 32×32. They are
  not general-purpose recognisers, and presenting them as such would be overclaiming — the
  first item on the Week-10 "common pitfalls" list.

### Future work

1. **Progressive fine-tuning** — unfreeze stages gradually with a discriminative
   (layer-wise decreasing) learning rate; the clearest remaining accuracy headroom.
2. **Train at native resolution with a modern small architecture** — a ResNet-20/WideResNet
   designed for 32×32 avoids the upsampling waste entirely and is the correct baseline this
   study's `Simple_CNN_Native32` variant gestures at.
3. **Stronger augmentation** — MixUp, CutMix, RandAugment; the current policy is mild, and
   the near-zero generalisation gaps suggest capacity to spare.
4. **Learning-rate schedules** — cosine annealing with warm restarts instead of
   plateau-triggered reduction, given §7.2's finding that LR is the highest-leverage knob.
5. **Proper uncertainty** — 5 seeds × 5 folds for every headline model, so all comparisons
   carry confidence intervals rather than only the pairwise significance tests in §5.5.
6. **Deployment path** — quantisation-aware training rather than post-training quantisation
   (§8.4), plus on-device benchmarking on the actual target hardware.

---

## 10. Appendices

### Appendix A — Deliverable manifest

| File | Description |
|---|---|
| `CIFAR10_CNN_Capstone.ipynb` | Fully-executed Jupyter notebook |
| `cnn_lab_results.json` | Structured results data (required) |
| `cnn_lab_comparison.csv` | Comparison table (required) |
| `Analysis_Report.md` / `.pdf` | This document |
| `Model_Performance_Summary.md` / `.pdf` | Executive summary |
| `capstone_lib.py` | Shared library — single code path for notebook and drivers |
| `run_main.py`, `run_features.py`, `run_paramtests.py`, `run_challenges.py` | Experiment drivers |
| `scripts_build_csv.py` | Dataset → CSV conversion |
| `data/train.csv`, `data/test.csv` | 50,000 × 3,073 and 10,000 × 3,073 dataset deliverables |
| `figures/` | All generated figures |
| `artifacts/` | Weights, per-epoch logs, raw metrics, t-SNE embeddings |

### Appendix B — Reproduction

```bash
python scripts_build_csv.py     # rebuild train.csv / test.csv from the archive
python run_main.py              # train + evaluate the four headline models
python run_features.py          # feature extraction + t-SNE
python run_paramtests.py        # 13-run parameter-modification study
python run_challenges.py        # fine-tuning, CV, ensembles, compression, latency
jupyter nbconvert --to notebook --execute CIFAR10_CNN_Capstone.ipynb
python make_reports.py          # regenerate this report and the summary
````
````